

HƯỚNG DẪN THỰC HIỆN ĐỒ ÁN MÔN HỌC

NT549 - Học máy tăng cường cho các hệ thống mạng

Giảng viên: TS. Huỳnh Văn Đăng, UIT-VNUHCM

Cập nhật: ngày 09/03/2026.

1. Giới thiệu

Tài liệu này cung cấp hướng dẫn chi tiết cho sinh viên thực hiện đồ án trong môn học Reinforcement Learning for Networked Systems. Mục tiêu của tài liệu là giúp sinh viên hiểu rõ quy trình triển khai một dự án học máy tăng cường áp dụng cho các bài toán trong hệ thống mạng, từ bước mô hình hóa bài toán đến triển khai thuật toán, đánh giá kết quả và trình bày báo cáo.

Đồ án tập trung vào việc giúp sinh viên:

- Hiểu cách mô hình hóa các vấn đề ra quyết định trong hệ thống mạng
- Áp dụng Reinforcement Learning để giải quyết các bài toán tối ưu
- Xây dựng môi trường mô phỏng
- Triển khai và huấn luyện RL agents
- Phân tích và đánh giá kết quả

2. Mục tiêu của đồ án

Sau khi hoàn thành đồ án, sinh viên cần đạt được các năng lực sau:

- Hiểu cách biểu diễn một bài toán mạng dưới dạng Markov Decision Process (MDP)
- Thiết kế và triển khai môi trường RL
- Áp dụng các thuật toán RL để giải bài toán
- Phân tích hành vi của agent
- Đánh giá hiệu năng hệ thống thông qua các chỉ số đo lường

3. Quy trình thực hiện đồ án

Sinh viên cần thực hiện đồ án theo quy trình sau:

1. Xác định bài toán mạng
2. Mô hình hóa bài toán thành MDP
3. Xây dựng môi trường RL
4. Triển khai thuật toán RL
5. Huấn luyện agent
6. Đánh giá và phân tích kết quả

4. Lựa chọn bài toán

Sinh viên cần chọn một vấn đề ra quyết định trong hệ thống mạng. Một số bài toán phổ biến gồm:

4.1 Network Routing

Agent học cách chọn đường đi tối ưu cho các gói tin nhằm giảm congestion hoặc giảm độ trễ.

4.2 Wireless Resource Allocation

Agent phân bổ tài nguyên vô tuyến như:

- kênh truyền
- công suất
- băng thông

4.3 Edge Computing

Agent quyết định:

- offload task
- xử lý local
- lựa chọn edge server

4.4 Network Traffic Control

Agent điều khiển lưu lượng mạng để tối ưu hiệu năng hệ thống.

5. Mô hình hóa bài toán Reinforcement Learning

Một bài toán Reinforcement Learning được biểu diễn dưới dạng Markov Decision Process gồm:

- State (trạng thái hệ thống)
- Action (hành động của agent)
- Reward (phần thưởng)
- Transition (chuyển trạng thái)
- Discount factor

5.1 State

State mô tả tình trạng của hệ thống mạng tại thời điểm t .

Ví dụ:

- queue length
- link utilization
- channel quality
- traffic demand

5.2 Action

Action là quyết định điều khiển của agent.

Ví dụ:

- chọn route
- phân bổ bandwidth
- chọn server xử lý

5.3 Reward

Reward phản ánh mục tiêu tối ưu của hệ thống.

Ví dụ:

- tăng throughput
- giảm delay
- giảm packet loss

Reward có thể được thiết kế dưới dạng hàm kết hợp nhiều mục tiêu.

6. Xây dựng môi trường mô phỏng

Sinh viên cần xây dựng một môi trường mô phỏng để RL agent tương tác.

Environment cần bao gồm:

- State space
- Action space
- Reward function
- Transition dynamics

Khuyến nghị sử dụng Gymnasium API để xây dựng environment.

6.1 Các thành phần tối thiểu của environment

Một environment chuẩn thường cần các thành phần sau:

- `__init__()` để khai báo tham số hệ thống, state space và action space
- `reset()` để khởi tạo lại trạng thái đầu mỗi episode
- `step(action)` để thực hiện một hành động và trả về trạng thái mới
- hàm hỗ trợ nội bộ để tính reward, cập nhật trạng thái, kiểm tra điều kiện dừng

Trong bài toán mạng, environment nên phản ánh được logic vận hành của hệ thống, ví dụ:

- gói tin đến theo thời gian
- queue tăng hoặc giảm

- link có giới hạn dung lượng
- reward phụ thuộc vào delay, throughput hoặc packet loss

6.2 Ví dụ minh họa chi tiết

Ví dụ dưới đây mô tả một environment đơn giản cho bài toán chọn đường truyền giữa 2 link. Agent quan sát độ dài hàng đợi của 2 link và chọn 1 link để gửi lưu lượng. Mục tiêu là giảm tắc nghẽn và cân bằng tải.

```
import numpy as np
import gymnasium as gym
from gymnasium import spaces

class SimpleRoutingEnv(gym.Env):
    """
    Môi trường RL đơn giản cho bài toán routing.

    State:
        [queue_1, queue_2, incoming_packets]

    Action:
        0 -> gửi toàn bộ lưu lượng mới vào link 1
        1 -> gửi toàn bộ lưu lượng mới vào link 2

    Reward:
        reward cao khi tổng queue nhỏ và cân bằng tải tốt
    """

    metadata = {"render_modes": ["human"]}

    def __init__(self, max_queue=20, max_steps=50, seed=None):
        super().__init__()

        self.max_queue = max_queue
        self.max_steps = max_steps
        self.current_step = 0

        # Mỗi state gồm 3 phần tử: queue_1, queue_2, incoming_packets
        self.observation_space = spaces.Box(
            low=np.array([0, 0, 0], dtype=np.float32),
            high=np.array([max_queue, max_queue, 10],
                           dtype=np.float32),
            dtype=np.float32,
        )

        # 2 hành động: chọn link 1 hoặc link 2
        self.action_space = spaces.Discrete(2)

        self.rng = np.random.default_rng(seed)
```

```

self.queue_1 = 0
self.queue_2 = 0
self.incoming_packets = 0
self.state = None

def reset(self, seed=None, options=None):
    """
    Khởi tạo lại môi trường ở đầu mỗi episode.
    """
    super().reset(seed=seed)
    if seed is not None:
        self.rng = np.random.default_rng(seed)

    self.current_step = 0
    self.queue_1 = self.rng.integers(0, 5)
    self.queue_2 = self.rng.integers(0, 5)
    self.incoming_packets = self.rng.integers(1, 6)

    self.state = np.array(
        [self.queue_1, self.queue_2, self.incoming_packets],
        dtype=np.float32,
    )

    info = {}
    return self.state, info

def step(self, action):
    """
    Thực hiện một hành động và cập nhật hệ thống.
    """
    assert self.action_space.contains(action), "Invalid action"

    self.current_step += 1

    # 1. Agent phân bổ lưu lượng mới vào một trong hai link
    if action == 0:
        self.queue_1 += self.incoming_packets
    else:
        self.queue_2 += self.incoming_packets

    # 2. Mỗi link phục vụ được một số gói tin ngẫu nhiên ở mỗi bước
    served_1 = self.rng.integers(1, 4)
    served_2 = self.rng.integers(1, 4)

    self.queue_1 = max(0, self.queue_1 - served_1)
    self.queue_2 = max(0, self.queue_2 - served_2)

    # 3. Lưu lượng mới đến ở bước tiếp theo
    self.incoming_packets = self.rng.integers(1, 6)

    # 4. Giới hạn queue tối đa để phản ánh buffer hữu hạn

```

```

overflow = 0
if self.queue_1 > self.max_queue:
    overflow += self.queue_1 - self.max_queue
    self.queue_1 = self.max_queue
if self.queue_2 > self.max_queue:
    overflow += self.queue_2 - self.max_queue
    self.queue_2 = self.max_queue

# 5. Tính reward
total_queue = self.queue_1 + self.queue_2
imbalance = abs(self.queue_1 - self.queue_2)

reward = 0.0
reward -= 0.1 * total_queue           # phạt khi queue lớn
reward -= 0.05 * imbalance            # phạt khi mất cân bằng tải
reward -= 1.0 * overflow              # phạt mạnh nếu tràn hàng
đợi

# 6. Tạo trạng thái kế tiếp
self.state = np.array(
    [self.queue_1, self.queue_2, self.incoming_packets],
    dtype=np.float32,
)

# 7. Điều kiện kết thúc episode
terminated = False
truncated = self.current_step >= self.max_steps

info = {
    "queue_1": self.queue_1,
    "queue_2": self.queue_2,
    "incoming_packets": self.incoming_packets,
    "overflow": overflow,
    "total_queue": total_queue,
}

return self.state, reward, terminated, truncated, info

def render(self):
    """
    Hiển thị trạng thái hiện tại của hệ thống.
    """
    print(
        f"Step={self.current_step}, "
        f"queue_1={self.queue_1}, "
        f"queue_2={self.queue_2}, "
        f"incoming={self.incoming_packets}"
    )

```

6.3 Giải thích chi tiết từng phần

a) Observation space

```
self.observation_space = spaces.Box(  
    low=np.array([0, 0, 0], dtype=np.float32),  
    high=np.array([max_queue, max_queue, 10], dtype=np.float32),  
    dtype=np.float32,  
)
```

Phần này định nghĩa không gian trạng thái. Ở đây trạng thái gồm ba biến liên tục/rời rạc biểu diễn dưới dạng vector số thực:

- độ dài hàng đợi của link 1
- độ dài hàng đợi của link 2
- số gói tin mới đến

b) Action space

```
self.action_space = spaces.Discrete(2)
```

Agent có hai hành động:

- chọn link 1
- chọn link 2

Nếu bài toán lớn hơn, action có thể là:

- chọn nhiều đường đi
- phân bổ một mức băng thông cụ thể
- chọn một edge server trong nhiều server

c) Hàm reset()

`reset()` được gọi ở đầu mỗi episode để đưa hệ thống về trạng thái ban đầu. Trạng thái khởi tạo có thể:

- cố định nếu muốn kiểm soát thí nghiệm
- ngẫu nhiên nếu muốn agent học tổng quát hơn

d) Hàm step(action)

Đây là phần quan trọng nhất. Trong một bước mô phỏng, environment thường thực hiện chuỗi logic sau:

1. nhận action từ agent
2. cập nhật hệ thống theo action
3. mô phỏng biến động ngẫu nhiên của môi trường

4. tính reward
5. trả về next state

e) Reward design

Trong ví dụ trên:

```
reward -= 0.1 * total_queue
reward -= 0.05 * imbalance
reward -= 1.0 * overflow
```

Reward được thiết kế để agent học cách:

- giữ tổng queue thấp
- cân bằng tải giữa các link
- tránh overflow

Đây là một ví dụ rất điển hình trong networked systems: reward không nên chỉ phản ánh một mục tiêu duy nhất, mà thường là sự cân bằng giữa nhiều tiêu chí hiệu năng.

6.4 Cách chạy thử environment

Sinh viên có thể kiểm tra environment trước khi kết nối với RL agent bằng đoạn mã sau:

```
env = SimpleRoutingEnv()
state, info = env.reset()
print("Initial state:", state)

for _ in range(5):
    action = env.action_space.sample() # chọn action ngẫu nhiên
    next_state, reward, terminated, truncated, info =
env.step(action)
    print("Action:", action)
    print("Next state:", next_state)
    print("Reward:", reward)
    print("Info:", info)
    print("-" * 40)

    if terminated or truncated:
        break
```

Bước này rất quan trọng vì nó giúp kiểm tra:

- state có cập nhật đúng không
- action có tác động đúng không
- reward có hợp lý không
- episode có dừng đúng không

6.5 Khuyến nghị khi xây dựng environment cho đồ án

Sinh viên nên bắt đầu từ môi trường đơn giản, sau đó mới tăng độ phức tạp. Một lộ trình hợp lý là:

- phiên bản 1: state và action đơn giản, reward dễ hiểu
- phiên bản 2: thêm traffic ngẫu nhiên hoặc ràng buộc hệ thống
- phiên bản 3: thêm nhiều mục tiêu tối ưu hoặc nhiều agents

Khi thiết kế environment, cần ưu tiên:

- logic hệ thống rõ ràng
- reward bám sát mục tiêu bài toán
- state đủ thông tin nhưng không quá phức tạp
- có baseline đơn giản để so sánh

6.6 Những lỗi phổ biến khi viết environment

- state không phản ánh đủ thông tin cần thiết
- action được định nghĩa không đúng với khả năng điều khiển thực tế
- reward thay đổi quá nhiều khiến agent khó học
- logic trong `step()` không nhất quán giữa các bước
- episode quá ngắn hoặc quá dài

Nếu environment được xây dựng tốt, quá trình huấn luyện RL phía sau sẽ ổn định và dễ phân tích hơn rất nhiều.

7. Triển khai thuật toán Reinforcement Learning

Sinh viên có thể sử dụng các thuật toán RL phổ biến như:

7.1 Tabular Reinforcement Learning

- Q-learning
- SARSA

7.2 Deep Reinforcement Learning

- Deep Q Network (DQN)

Các kỹ thuật ổn định training gồm:

- experience replay
- target network

7.3 Policy Gradient

Policy Gradient trực tiếp học policy thay vì value function.

7.4 Actor–Critic

Kết hợp policy network và value network.

Ví dụ thuật toán:

- A2C
- PPO

8. Huấn luyện Agent

Training RL agent được thực hiện qua nhiều episode.

Quy trình training cơ bản:

1. Reset environment
2. Agent chọn action
3. Environment trả về reward và next state
4. Agent cập nhật policy
5. Lặp lại cho đến khi hội tụ

Các thông số training quan trọng gồm:

- learning rate
- discount factor
- batch size
- exploration rate

9. Đánh giá kết quả

Sinh viên cần đánh giá hiệu năng của RL agent thông qua các chỉ số sau:

- cumulative reward
- throughput
- delay
- packet loss

Ngoài ra cần so sánh RL với các phương pháp baseline như:

- random policy
- heuristic algorithm
- greedy strategy

10. Visualization kết quả

Sinh viên cần trực quan hóa kết quả training bằng các biểu đồ như:

- training reward curve
- convergence curve
- performance comparison

Khuyến nghị sử dụng thư viện matplotlib.

11. Video demo

Ngoài báo cáo và source code, sinh viên cần nộp một video demo (3–5 phút).

Video cần thể hiện các nội dung sau:

11.1 Giới thiệu bài toán

- mô tả hệ thống mạng
- mục tiêu tối ưu

11.2 Demo chương trình

Hiển thị quá trình training hoặc chạy chương trình.

11.3 Minh họa quyết định của Agent

Video cần thể hiện rõ cách agent ra quyết định, ví dụ:

- chọn route
- phân bổ tài nguyên
- offload task

Có thể hiển thị dưới dạng:

- animation
- network visualization
- console output

11.4 Visualization

Video nên hiển thị các biểu đồ training hoặc hành vi của agent.

12. Cấu trúc báo cáo đồ án

Báo cáo cần được viết bằng tiếng Việt và nên có độ dài khoảng 6–8 trang.

1. Giới thiệu

Trình bày bối cảnh của bài toán, động lực nghiên cứu và mục tiêu của đề án.

2. Mô hình hệ thống

Mô tả topology mạng, các thành phần của hệ thống và các giả định được sử dụng.

3. Mô hình Reinforcement Learning

Trình bày cách biểu diễn bài toán dưới dạng RL bao gồm:

- state
- action
- reward
- mục tiêu tối ưu

4. Thuật toán đề xuất

Trình bày thuật toán RL được sử dụng và cách triển khai.

5. Thiết lập mô phỏng

Mô tả môi trường mô phỏng và các tham số được sử dụng trong thí nghiệm.

6. Kết quả và đánh giá

Trình bày các kết quả thực nghiệm và so sánh với baseline.

7. Thảo luận

Phân tích hành vi của agent và thảo luận về ưu điểm, hạn chế của phương pháp.

8. Kết luận

Tóm tắt những đóng góp chính của đề án.

13. Các lỗi thường gặp

Một số lỗi phổ biến khi thực hiện đề án:

- thiết kế reward function chưa hợp lý
- state representation quá phức tạp
- training chưa đủ episode
- thiếu baseline comparison

Sinh viên cần kiểm tra kỹ các yếu tố này trong quá trình triển khai.

14. Yêu cầu nộp bài

Sinh viên cần nộp các thành phần sau:

- Source code
- Báo cáo PDF
- Slide trình bày
- Video demo

Tất cả các file cần được nộp đúng thời hạn theo hướng dẫn của giảng viên.

Kết luận

Đồ án *Học máy tăng cường cho các hệ thống mạng* giúp sinh viên áp dụng các kỹ thuật học máy hiện đại để giải quyết các vấn đề trong hệ thống mạng. Thông qua quá trình triển khai, sinh viên sẽ hiểu sâu hơn về cách RL agents học và ra quyết định trong các môi trường phức tạp.